# 第四章 系统实现与调试

第三章完成了基于 RK3588 嵌入式平台的目标检测系统总体设计，本章在此基础上进一步说明系统的具体实现过程。系统实现围绕两个核心目标展开：一是将训练得到的无人机检测模型转换为 RK3588 NPU 可加载的 RKNN 模型，并完成板端推理；二是在板端构建固定视频检测和实时 RTSP 推流两条运行路径，使系统既能够进行可重复实验，又能够满足实时演示需求。

本章所述实现主要包括开发环境配置、模型导出与转换、RKNN 推理封装、固定视频检测程序、实时 RTSP 检测程序、轻量跟踪与动态 ROI 策略、多 context 并行推理以及运行日志记录等内容。与纯算法训练不同，嵌入式部署需要同时处理模型格式、输入输出张量、视频解码、线程同步、推流显示和性能统计等工程问题。因此，本章重点从工程实现角度说明系统各模块如何协同工作，并为第五章的实验测试和性能分析提供实现基础。

## 4.1 开发与部署环境

系统开发过程分为 PC/WSL 端和 RK3588 板端两部分。PC/WSL 端主要用于模型训练、模型导出、RKNN 转换准备和代码整理；RK3588 板端主要用于实际模型加载、视频输入、NPU 推理、检测结果绘制和 RTSP 输出。这样的分工可以降低板端开发压力，使耗时较长的模型训练和格式转换尽量在 PC 端完成，而把板端工作集中在实时推理和系统集成上。

表 4.1 给出了本文系统实现所涉及的主要软硬件环境。

表 4.1 系统开发与部署环境

| 类别 | 环境或工具 | 作用说明 |
| --- | --- | --- |
| 开发主机 | Windows 与 WSL 环境 | 用于代码管理、模型训练脚本运行、ONNX/RKNN 转换准备和论文资料整理 |
| 嵌入式平台 | RK3588 开发板 | 作为最终部署平台，利用板载 NPU 执行目标检测推理 |
| 目标检测模型 | YOLOv10 单类无人机检测模型 | 作为系统目标检测算法主体，输出无人机候选框和置信度 |
| 推理框架 | RKNN Runtime | 在 RK3588 板端加载 RKNN 模型并调用 NPU 推理 |
| 视频处理库 | OpenCV | 用于视频文件读取、USB 摄像头读取、图像缩放、颜色转换和检测框绘制 |
| 推流组件 | GStreamer RTSP Server | 用于将检测后的实时画面发布为 RTSP 视频流 |
| 构建工具 | CMake 与 Make | 用于在 RK3588 板端编译 C/C++ 程序 |
| 实验记录 | CSV、JSONL 和运行日志 | 用于保存检测结果、ROI 信息、FPS、延迟和阶段耗时 |

在板端运行时，系统采用 C/C++ 作为主要实现语言。其原因在于 C/C++ 便于直接调用 RKNN Runtime、OpenCV 和 GStreamer 等底层库，能够更好地控制线程、队列和内存使用。同时，C/C++ 程序在嵌入式平台上运行开销较低，更适合实时视频处理场景。

## 4.2 模型导出与 RKNN 转换实现

本文使用的无人机检测模型首先在 PC/WSL 环境中完成训练，得到 PyTorch 权重文件。为了在 RK3588 上运行，需要将模型从 PyTorch 格式转换为 ONNX 格式，再由 RKNN Toolkit2 转换为 RKNN 模型。整体流程为：

```
best.pt -> best.onnx -> best.rk3588.fp.rknn -> RK3588 NPU 推理
```

在模型转换过程中，本文优先采用 FP 类型 RKNN 模型作为稳定部署版本。虽然 INT8 量化理论上可以进一步降低计算和存储开销，但量化需要校准数据集、量化误差评估和板端精度验证。当前系统已经稳定跑通 FP 模型，因此本章将 FP RKNN 作为主实现路径，INT8 量化仅作为后续优化方向保留，不将其写成已经完整验证的结果。

表 4.2 给出了模型迁移过程中涉及的主要文件类型及其作用。

表 4.2 模型文件及作用

| 文件类型 | 示例文件 | 作用说明 |
|---|---|---|
| PyTorch 权重 | best.pt | 训练完成后得到的原始模型权重，用于继续训练、验证和导出 |
| ONNX 模型 | best.onnx | 用于跨框架转换的中间模型格式，便于导入 RKNN Toolkit2 |
| RKNN 模型 | best.rk3588.fp.rknn | RK3588 板端实际加载的模型文件，由 RKNN Runtime 调用 |
| 检测输出 | [1, 5, 8400] | 单类别无人机检测模型的输出张量，包含边界框参数和置信度 |

为了适配单类无人机检测任务，后处理模块不再使用 COCO 多类别标签表作为唯一判断依据，而是根据模型输出通道数识别当前模型类别数。当输出张量为 [1, 5, 8400] 时，系统将其解析为单类别检测输出，其中 5 个通道分别对应候选框中心点、宽高和置信度信息。检测结果在可视化时统一标注为 drone，避免出现类别映射错误。

## 4.3 RKNN 推理模块实现

RKNN 推理模块是系统的核心计算模块，负责完成模型加载、输入预处理、NPU 推理、输出读取和检测结果解析。本文将该部分封装为独立的检测器类，使固定视频检测程序和实时 RTSP 程序可以复用相同的推理逻辑。

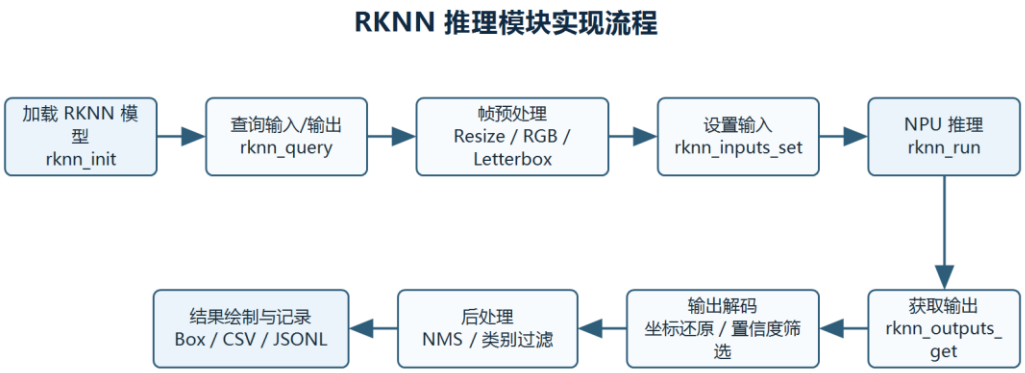


图 4.1 RKNN 推理模块实现流程

RKNN 推理模块的主要步骤如下：

1. 调用 rknn_init 加载 RKNN 模型文件；
2. 通过 rknn_query 查询模型输入输出张量属性；
3. 根据输入张量格式确定模型输入宽度、高度、通道数和数据排布；
4. 对原始图像进行缩放、补边和颜色空间转换；
5. 调用 rknn_inputs_set 设置模型输入；
6. 调用 rknn_run 在 NPU 上执行模型推理；
7. 调用 rknn_outputs_get 获取输出张量；
8. 对输出张量执行候选框解析、置信度过滤和非极大值抑制；
9. 将检测框坐标映射回原始图像尺寸；
10. 释放输出内存并返回检测结果。

在实现中，系统会在模型加载阶段打印输入输出张量属性，包括张量维度、数据类型和存储格式。这样做的目的是在模型转换后第一时间发现输入尺寸不匹配、输出通道数异常或数据布局不一致等问题。对于 YOLO 系列模型而言，输出张量格

式可能随导出参数而变化，如果不在板端进行显式检查，容易导致后处理阶段解析错误。

表 4.3 给出了 RKNN 推理模块中几个关键阶段的功能划分。

表 4.3 RKNN 推理模块关键阶段

| 阶段 | 对应功能 | 实现重点 |
| --- | --- | --- |
| 模型初始化 | 加载 RKNN 文件并查询张量属性 | 判断输入尺寸、通道数和输出布局是否符合预期 |
| 图像预处理 | 缩放、补边、颜色转换 | 保持与训练和导出阶段一致的输入格式 |
| 输入设置 | 将预处理后的图像写入 RKNN 输入 | 当前稳定路径使用常规输入设置方式 |
| NPU 推理 | 执行 rknn_run | 由 RK3588 NPU 完成主要神经网络计算 |
| 输出解析 | 候选框解码、阈值筛选和 NMS | 支持单类无人机检测输出格式 |
| 坐标还原 | 将模型坐标映射回原图 | 保证绘制框位置与原始画面一致 |

为了便于后续性能分析，推理模块还提供了可选的阶段耗时统计接口。开启 profiling 后，系统可以分别记录图像准备、输入设置、NPU 执行、输出读取、解码和释放等阶段耗时。该接口在正常演示时默认关闭，以减少日志输出和额外开销；在实验分析时再通过环境变量开启。

## 4.4 固定视频检测程序实现

固定视频检测程序 rk_yolo_video 是本文最先跑通的 RK3588 验证路径。该程序的作用是在板端读取本地视频文件，逐帧执行 RKNN 推理，将检测框绘制到原始画面中，并输出带框视频、检测结果 CSV 和 ROI JSONL 文件。

该程序的命令行接口如下：

```
./rk_yolo_video <input_video> <output_video> [model_path] [score_thresh]
[nms_thresh] [detections_csv] [roi_jsonl]
```

其中，input_video 为输入视频文件，output_video 为带框输出视频，model_path 为 RKNN 模型路径，score_thresh 和 nms_thresh 分别为置信度阈值和非极大值抑制

阈值。如果没有显式指定 CSV 和 JSONL 文件路径，程序会根据输出视频文件名自动生成对应的检测记录文件。

固定视频检测程序的输出包括三类数据：

1. 带框视频，用于直观看到模型是否检测到目标；
2. CSV 检测记录，用于统计每一帧的检测结果；
3. ROI JSONL 文件，用于兼容后续动态 ROI 或旧系统输出格式。

CSV 文件每一行对应一个检测框，字段包括帧序号、类别编号、类别名称、置信度、左上角坐标、宽度和高度。其字段格式如下：

```
frame_index,class_id,class_name,score,left,top,width,height
```

ROI JSONL 文件则按帧保存检测结果，每一行是一个 JSON 对象，便于后续程序按帧读取目标区域信息。固定视频路径具有较强的可重复性，同一段视频、同一模型和同一阈值可以反复运行得到可比较结果，因此本文将其作为板端固定输入实验的主要入口。

## 4.5 实时 RTSP 检测程序实现

在固定视频检测路径稳定后，本文进一步实现了实时 RTSP 检测程序 rk_yolo_live_rtsp。该程序既支持 USB 摄像头输入，也支持本地视频文件循环输入。系统读取视频帧后执行目标检测，将检测框叠加到画面上，再通过 GStreamer RTSP Server 发布为 RTSP 视频流，使 PC 端可以使用播放器或 GStreamer 客户端实时查看检测画面。

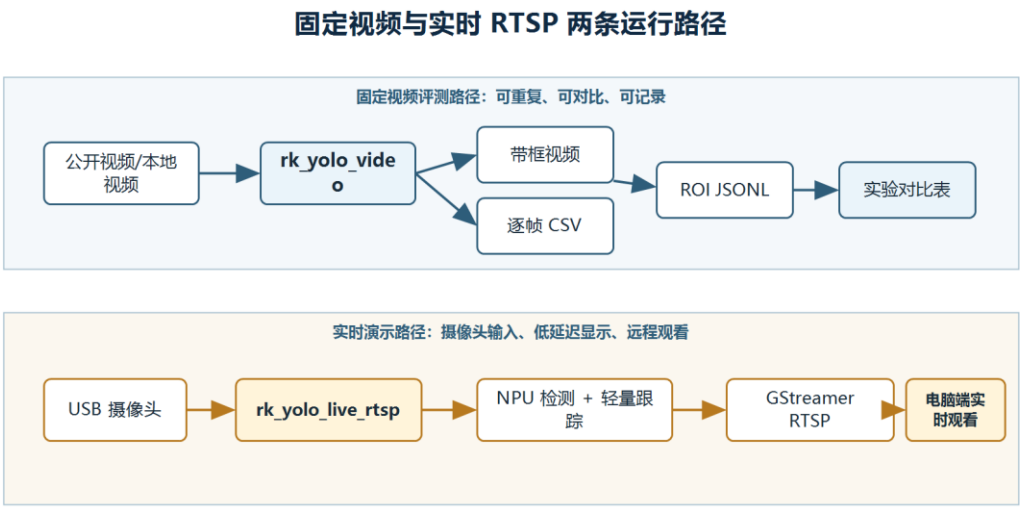


图 4.2 固定视频与实时 RTSP 两条运行路径

实时 RTSP 程序的基本命令行接口如下：

```
./rk_yolo_live_rtsp [device] [model] [mount] [width] [height] [fps] [score]
[nms] [port] [detect_every_n]
```

其中，device 可以是 /dev/video* 摄像头设备，也可以是本地视频文件路径；model 为 RKNN 模型路径；mount 为 RTSP 挂载路径；width、height 和 fps 用于指定输出视频参数；score 和 nms 用于设置检测阈值；detect_every_n 用于控制每隔多少帧执行一次完整 NPU 检测。

实时 RTSP 程序内部采用三阶段流水线结构：

1. 采集线程负责读取摄像头或视频文件，并将最新帧写入有界队列；
2. 推理线程负责从队列中取出图像，根据调度策略执行 NPU 检测或复用跟踪结果；
3. 发布线程负责绘制检测框，将画面推送给 RTSP 服务端。

采用有界队列的原因是实时视频系统不能无限积压历史帧。如果推理速度低于输入帧率，系统应优先保证输出画面接近当前时刻，而不是依次处理所有过期帧。因此，队列长度被设计得较小，在必要时丢弃旧帧，以降低端到端延迟。

RTSP 输出基于 GStreamer 实现。程序内部创建 appsrc，将绘制后的图像帧推入 GStreamer 管线，再通过 H.264 编码器和 RTSP Server 对外发布。PC 端可以通过如下地址查看检测画面：

```
rtsp://<board-ip>:8554/drone
```

对于固定视频循环演示，可以设置 RK_YOLO_INPUT_LOOP=1，使程序在视频播放结束后自动回到开头，便于长时间观察和演示。在 USB 摄像头输入场景下，程序还支持对特定 UVC 摄像头执行启动参数调整，例如设置焦距、关闭自动对焦和设置变焦值，以提高画面稳定性。

## 4.6 检测间隔、动态 ROI 与轻量跟踪实现

在实时检测场景中，如果每一帧都执行完整 NPU 推理，系统可能出现延迟升高或队列积压。为此，本文实现了检测间隔策略，即通过 detect_every_n 参数控制完整检测频率。当 detect_every_n=1 时，系统每帧执行一次 NPU 检测；当该参数大于 1 时，系统仅在关键帧执行完整检测，其余帧根据轻量跟踪结果更新显示框。

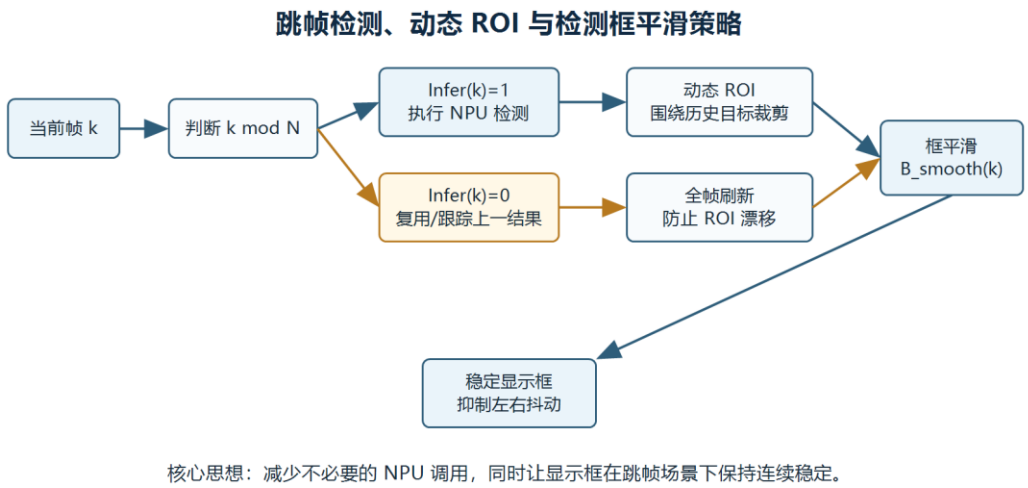


图 4.3 跳帧检测、动态 ROI 与检测框平滑策略

检测间隔策略可表示为：

$$\text{Infer}(k)=1,\ k \bmod N = 0;\ \text{Infer}(k)=0,\ k \bmod N \neq 0 \tag{4.1}$$

其中，k 表示当前帧序号，N 表示检测间隔。当 Infer(k)=1 时，系统执行完整 NPU 检测；当 Infer(k)=0 时，系统不调用 NPU，而是使用跟踪或上一帧检测结果进行显示更新。

为了减少跳帧检测导致的检测框抖动，系统实现了轻量框平滑策略。其基本思想是将当前检测框与上一帧显示框进行加权融合：

$$B_smooth(k)=alpha\ B_detect(k)+(1-alpha)B_smooth(k-1) \tag{4.2}$$

其中，B_detect(k) 表示当前检测框，B_smooth(k-1) 表示上一帧平滑后的显示框，alpha 为平滑系数。alpha 越大，显示框越接近当前检测结果；alpha 越小，显示框越稳定但响应速度越慢。系统通过环境变量提供该参数，使不同场景下可以根据目标运动速度和画面抖动程度进行调节。

除框平滑外，系统还实现了动态 ROI 策略。当上一帧已经检测到目标时，下一次关键帧推理可以优先围绕历史目标区域构造裁剪窗口，并加入一定边界扩展。这样可以在目标较小的情况下提高局部区域内的目标占比，同时减少无关背景对检测的干扰。为了避免 ROI 长时间漂移，系统设置了周期性全帧刷新机制，使检测过程能够重新回到完整画面中恢复目标。

表 4.4 给出了实时检测策略相关的主要配置项。

表 4.4 实时检测策略配置项

| 配置项 | 默认或推荐含义 | 作用说明 |
| --- | --- | --- |
| detect_every_n | 控制完整检测间隔 | 在 NPU 调用频率和显示稳定性之间折中 |
| RK_YOLO_TRACK_MODE | 轻量运动跟踪模式 | 在非关键帧估计检测框位置 |
| RK_YOLO_BOX_SMOOTH | 是否启用框平滑 | 减少检测框在相邻帧之间的突变 |
| RK_YOLO_BOX_SMOOTH_ALPHA | 平滑系数 | 控制检测框响应速度与稳定性的平衡 |
| RK_YOLO_DYNAMIC_ROI | 是否启用动态 ROI | 根据历史检测区域裁剪下一次推理输入 |
| RK_YOLO_ROI_REFRESH | ROI 刷新间隔 | 定期恢复全帧检测，降低跟踪漂移风险 |

## 4.7 多 context NPU 并行推理实现

为了满足硬件优化方向的研究需求，本文在实时 RTSP 程序中实现了实验性的多 context NPU 并行推理机制。该机制的核心思想是在程序中创建多个 RKNN 检测器实例，每个实例拥有独立的 RKNN context，并由多个推理工作线程并行处理不同帧。

多 context 模式仅在 detect_every_n=1 时启用。这一限制是有意设计的：当系统已经通过 detect_every_n>1 降低 NPU 调用频率时，多 context 并行推理的收益并不明显，反而可能增加调度复杂度。只有在每帧都需要完整检测时，多 context 才能更直接地体现 NPU 并行吞吐能力。

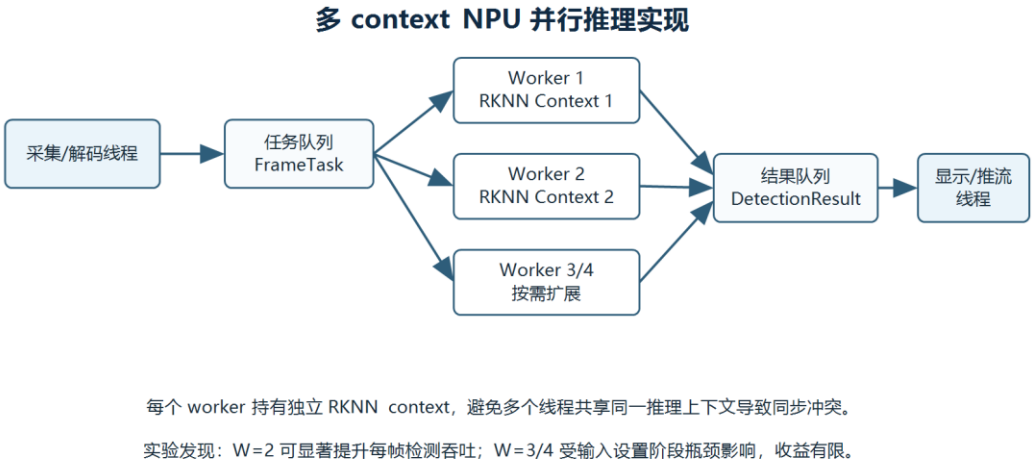


图 4.4 多 context NPU 并行推理实现

多 context 推理的实现流程如下：

1. 主线程读取 RK_YOLO_INFER_WORKERS 环境变量，确定推理工作线程数量；
2. 为每个工作线程创建独立的 RKNN 检测器和 RKNN context；
3. 采集线程将最新帧送入调度队列；
4. 调度逻辑将待检测帧分配给空闲 worker；
5. worker 完成 NPU 推理后将结果写入结果队列；
6. 发布线程根据帧序号和时间戳选择最新可用结果进行绘制。

表 4.5 给出了单 context 和多 context 实现方式的对比。

表 4.5 单 context 与多 context 实现对比

| 实现方式 | 特点 | 适用场景 |
| --- | --- | --- |
| 单 context | 资源占用较低，调度简单，延迟更容易控制 | 实时演示和稳定运行 |
| 双 context | 可以重叠多个推理任务，提高每帧检测吞吐 | NPU 并行能力验证 |
| 三 context 及以上 | 调度和内存开销进一步增加，收益需要实验确认 | 扩展性分析，不作为默认配置 |

多 context 机制使本文不仅能够完成“模型在 RK3588 上跑通”的基本目标，还能够围绕导师提出的 NPU 多线程方向开展实验。需要注意的是，多 context 的目标并不是简单替代当前实时演示配置，而是提供一条硬件并行优化路线。后续实验需要同时比较吞吐、延迟、CPU 占用和内存占用，才能判断其实际价值。

## 4.8 日志记录与性能调试接口

为了支撑论文实验分析，系统在实现过程中加入了多层日志记录机制。固定视频检测程序主要输出检测结果 CSV、ROI JSONL 和带框视频；实时 RTSP 程序则输出运行时 FPS、NPU 推理频率、端到端延迟、工作线程耗时和队列状态等信息。

实时程序的常规日志包括：

1. stream_fps：实际发布的视频帧率；
2. npu_fps：实际执行 NPU 推理的频率；
3. roi_fps：动态 ROI 推理频率；
4. work_ms：单帧主要处理耗时；
5. 队列丢帧信息：用于判断系统是否出现输入积压；
6. 当前配置项：包括检测间隔、平滑参数、worker 数量和 ROI 开关。

当需要进行更细粒度性能分析时，可以设置：

```
RK_YOLO_PROFILE=1
```

开启该开关后，程序会输出推理阶段的细分耗时，包括预处理、输入设置、NPU 运行、输出读取、检测解码和绘制等。通过这些指标，可以判断系统瓶颈究竟来自 NPU 计算、输入数据传输、后处理还是视频输出。

系统还保留了实验性的 zero-copy input 路径，对应环境变量为：

```
RK_YOLO_ZERO_COPY_INPUT=1
```

该路径用于探索 RKNN 输入内存绑定是否能够降低输入设置开销。但根据当前工程状态，zero-copy 仍属于实验功能，默认关闭。本文在系统实现章节中仅说明其作为调试接口存在，不将其作为稳定主路径。这样的写法可以真实反映项目进展，避免把尚未完整验证的优化方向写成成熟结果。

## 4.9 本章小结

本章围绕 RK3588 嵌入式平台上的目标检测系统实现过程进行了说明。首先介绍了 PC/WSL 端和 RK3588 板端的开发部署环境；随后说明了从 PyTorch 权重到 ONNX，再到 RKNN 模型的迁移过程，并明确当前稳定主路径为 FP RKNN 模型。接着，本章详细介绍了 RKNN 推理模块的封装方式，包括模型加载、输入预处理、NPU 推理、输出解析和坐标还原等步骤。

在系统运行层面，本章分别介绍了固定视频检测程序 rk_yolo_video 和实时 RTSP 检测程序 rk_yolo_live_rtsp 的实现。固定视频检测程序用于可重复实验和模型验证，实时 RTSP 程序用于实时观看和演示。针对实时场景中的延迟和检测框抖动问题，本章说明了检测间隔、轻量跟踪、框平滑和动态 ROI 等策略的实现方式。同时，为满足 NPU 多线程优化方向，本章实现了多 context 并行推理机制，并通过日志和 profiling 接口为后续实验分析提供数据基础。

通过本章实现，系统已经具备在 RK3588 板端运行无人机目标检测模型、输出带框视频、实时 RTSP 推流以及记录性能数据的能力。下一章将在此基础上，对固定视频检测、实时 RTSP 演示、检测间隔、框平滑和多 context 推理等配置进行实验测试与结果分析。